

CovenantIQ

AI-powered covenant compliance and credit portfolio intelligence. Reads loan and credit agreements with an LLM, tracks financial covenants against borrower statements over time, and answers portfolio questions in natural language — grounded in the original documents.

Python · FastAPI

LangChain

LLM structured extraction

RAG

PostgreSQL + pgvector

React

This is a personal portfolio project built to demonstrate applied LLM engineering for financial-services use cases. All figures, borrower names, and documents referenced in this document are synthetic. It is not affiliated with, and does not process data from, any real financial institution, lender, or client.

1 Problem

2 What it does

3 Architecture

4 Extraction pipeline

5 Compliance engine

6 Dashboard & reporting

7 Ask-your-portfolio (RAG)

8 Data model

9 Tech stack

10 Security & governance

11 Repository

12 Roadmap

01

The problem

Every credit agreement a lender signs — a term loan, a debenture, a revolving facility — comes with **financial covenants**: contractual thresholds the borrower must keep meeting, like a maximum Net Debt / EBITDA ratio, a minimum liquidity balance, or a minimum debt-service coverage ratio.

In most credit funds and bank portfolios, checking these covenants is still manual: an analyst re-reads the agreement to recall the exact definition and threshold, then cross-references it against whatever financial statement the borrower sent that quarter — a spreadsheet, a PDF, sometimes a scanned report. Across a portfolio of fifty or a hundred loans, this doesn't scale, and it means breaches are often caught **after** the fact, in the next scheduled review, instead of as a trend the moment the borrower's headroom starts shrinking.

CovenantIQ is a proof-of-concept for the other approach: read the legal document once with an LLM to get a structured, queryable definition of every covenant, then

feed financial data into that structure continuously so breaches — and early warning signs — surface automatically.

02

What CovenantIQ does

Four capabilities, each usable on its own, that compose into one pipeline: document in, portfolio risk view out.

EXTRACT

Read the agreement

Upload a credit agreement PDF. An LLM extraction chain pulls parties, principal, rate, maturity, and — the hard part — every financial covenant as a structured, typed definition (metric, operator, threshold, test frequency, cure period).

MONITOR

Track compliance over time

Each time a borrower's financial statement comes in, a second extraction pass pulls the line items each covenant needs, computes the actual ratio, and stores a dated compliance snapshot — not just pass/fail, but how much headroom is left.

VISUALIZE

See the whole portfolio

A dashboard rolls every loan's covenant status into one view: what's compliant, what's in warning range, what's breached, and whose headroom has been shrinking for three quarters running.

ASK

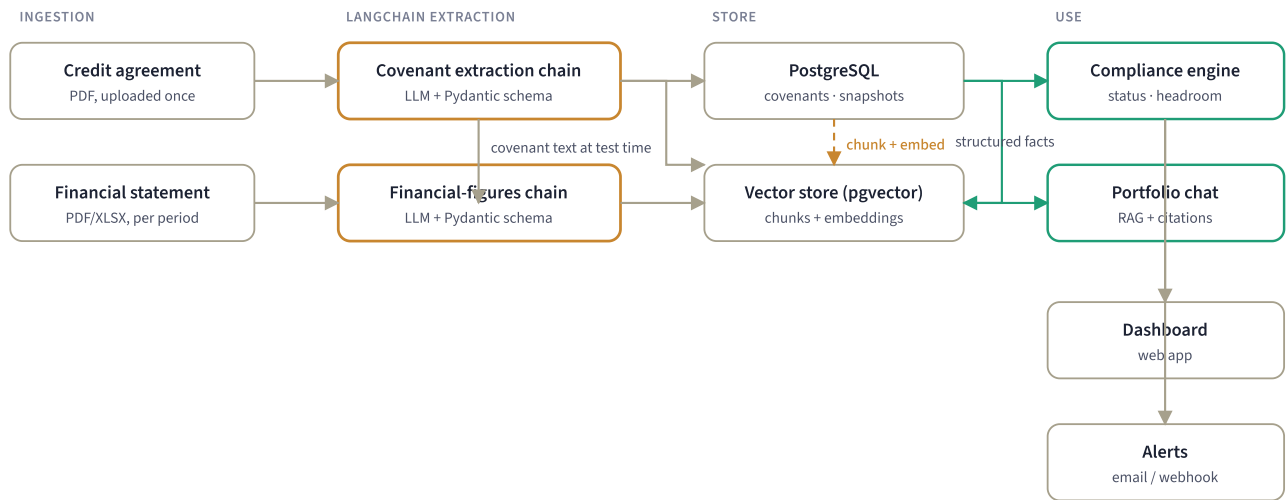
Query it in plain English

A retrieval-augmented chat answers questions like "which loans have a leverage covenant with less than 10% headroom?" — grounded in the actual agreement text, with a citation back to the source page.

03

Architecture

Two ingestion paths feed the same store: the agreement (read once, rarely changes) and the financial statements (read every reporting period). The compliance engine and the chat both read from that store — one computes, the other answers.



Two LangChain extraction chains feed one store — structured facts in Postgres, retrievable chunks in a vector store. Everything downstream (compliance engine, dashboard, alerts, chat) reads from that store; nothing re-parses the PDF.

04

The extraction pipeline

The hardest part of this project isn't calling an LLM — it's getting a covenant definition out of dense legal prose as something a program can actually test later.

Structured output, not free text. Every extraction call uses LangChain's `with_structured_output()` bound to a Pydantic model, so the LLM's response is validated against a schema before it ever reaches the database — a malformed covenant (missing threshold, ambiguous operator) fails loudly at extraction time, not three months later when the engine tries to test it.

```

class Covenant(BaseModel):
    name: str
    metric: Literal["net_debt_ebitda", "dscr", "min_liquidity", "min_equity", "current_ratio"]
    operator: Literal["lte", "gte"]
    threshold: float
    test_frequency: Literal["quarterly", "annual"]
    cure_period_days: int | None
    source_clause: str          # verbatim clause, for audit + citations
    source_page: int

    extraction_chain = (
        prompt_template
        | llm.with_structured_output(CovenantExtractionResult)
    )
    result = extraction_chain.invoke({"document_text": agreement_text})

```

Chunking with page anchors. The agreement is split into overlapping chunks that retain page numbers, so every extracted covenant carries a `source_page` and the exact clause it came from — the same anchor the chat's citations use later.

Two-pass extraction for financial statements. A first pass classifies which line items are even present (statements vary in format and language); a second, schema-locked pass extracts only the figures each active covenant in that loan actually needs — asking for less gets better accuracy than asking an LLM to extract "everything" from a financial statement.

Human-in-the-loop by design. Every extracted covenant is shown to a credit analyst before it goes live, with the source clause displayed side-by-side — this mirrors how the compliance engine later treats borderline cases (see §5): confidence, not blind automation.

05

Compliance engine

A config-driven rules engine, not a hardcoded one — the same principle that makes a portfolio's covenant mix (which varies loan to loan) something you configure, not something you redeploy for.

For every covenant, at every test date, the engine computes: `actual_value` from the latest financial snapshot, compares it against `threshold` using the stored `operator`, and derives a **headroom percentage** — how much room is left before breach, not just a boolean. A shrinking headroom trend across two or three consecutive periods is flagged as **early warning**, before an actual breach occurs.

Net Debt / EBITDA $\leq 3.00x$

● Compliant

Actual: 2.41x · Headroom: 20% · Trend: stable (3 periods)



Min. Liquidity $\geq$ R\$ 2.0M

● Early warning

Actual: R\$ 2.18M · Headroom: 9% · Trend: shrinking (3 periods)



DSCR $\geq 1.20x$

● Breach

Actual: 1.09x · Headroom: -9% · Cure period: 30 days remaining



Illustrative example — synthetic figures, not a real borrower.

Every computed snapshot is versioned and immutable: reprocessing a statement never overwrites history, so the trend line the engine flags is always reconstructable and auditable.

06

Dashboard & reporting

One portfolio-level view, three drill-downs.

- **Portfolio heatmap** — every loan $\times$ every covenant, compliant / warning / breach, sorted by worst status first.
- **Headroom trend charts** — per covenant, per loan, over its full test history — the early-warning signal made visible.
- **Upcoming test calendar** — which covenants come due for testing in the next 30/60/90 days, so nothing is reviewed late.
- **Board-ready export** — a generated PDF/Excel summary per loan or per portfolio, the same shape a credit committee memo needs today, built automatically instead of assembled by hand.

07

"Ask your portfolio" (RAG)

A retrieval-augmented chat over the same store the dashboard reads from — grounded, not a general-purpose chatbot bolted on top.

Every question runs a hybrid retrieval: a structured query against Postgres for anything quantitative ("which loans", "less than X headroom"), and a semantic search against the vector store for anything that requires reading the agreement's actual language ("summarize the change-of-control clause in Loan #114"). The LLM composes the final answer from both, and every claim it makes about a covenant links back to the `source_page` it came from — the same anchor captured at extraction time in §4.

```
retrieval_chain = (
    {"structured": covenant_sql_retriever, "context": vector_retriever}
    | answer_prompt
    | llm
    | citation_parser    # attaches source_page + agreement_id to every claim
)
retrieval_chain.invoke("Which loans have a leverage covenant under 10% headroom?")
```

This is the one feature in the whole pipeline built specifically to showcase LangChain's retrieval + chain-composition primitives — the extraction pipeline in §4 could run on the raw OpenAI/Anthropic SDKs just as well, and deliberately does, to show both approaches in the same codebase.

08

Data model

ENTITY	KEY FIELDS & NOTES
Borrower	Legal name, sector, synthetic identifier. No real-world PII in the demo dataset.
Loan	Principal, rate, maturity, agreement reference, status.
Covenant	Metric, operator, threshold, test frequency, cure period, source clause + page (see §4).
FinancialSnapshot	Period, extracted line items, source document reference.
ComplianceResult	Covenant × snapshot, actual value, headroom, status, computed at (immutable, versioned).
DocumentChunk	Agreement id, page, text, embedding vector — the retrieval unit for §7.
Alert	Trigger rule, covenant, fired at, acknowledged by.
User	Role (analyst, credit officer, admin), auth.

Tech stack

LAYER	CHOICE & WHY
API	FastAPI — async, typed, auto-generated OpenAPI docs.
LLM orchestration	LangChain (extraction chains with structured output, retrieval chains for §7) — plus one pipeline stage on the raw provider SDK, deliberately, to show both.
Database	PostgreSQL + pgvector — one database for both structured covenant data and embeddings, instead of a separate vector-only store.
Background jobs	Celery + Redis — extraction and re-scoring run async, off the request path.
Frontend	React + TypeScript , charts via a lightweight SVG-based library.
Auth	Role-based access (analyst / credit officer / admin), JWT sessions.
Deployment	Docker Compose for local/demo; a single Postgres instance is enough at portfolio scale (hundreds, not millions, of loans).

Security & governance considerations

- **Human-in-the-loop extraction** (§4) — no covenant goes live unreviewed; the source clause is always shown next to the parsed value.
- **Immutable compliance history** (§5, §8) — snapshots are append-only, so no one can silently "fix" a past breach.
- **Every AI-generated answer is grounded and cited** (§7) — the chat never states a covenant figure without a source page attached.
- **Role-based access** — read vs. write vs. admin, with an audit log on covenant edits and alert acknowledgements.
- **No real financial data anywhere in this repository** — the demo dataset is entirely synthetic, generated for this project.

Suggested repository structure

```
covenant-iq/
├─ api/                FastAPI app – routes, auth, schemas
│  └─ extraction/      LangChain chains (agreement + financials)
│  └─ engine/          compliance engine (config-driven rules)
│  └─ chat/            RAG retrieval chain
│  └─ models/          SQLAlchemY models
├─ worker/             Celery tasks (async extraction, re-scoring)
├─ web/                React + TypeScript dashboard
├─ data/synthetic/     generated demo loans, agreements, statements
├─ docs/
│  └─ whitepaper.pdf   this document
├─ docker-compose.yml
└─ README.md
```

Repository name: covenant-iq · alternates: covenant-sentinel,
creditlens-ai, loanwatch.

12

Roadmap

- **v0.1 — Core pipeline:** agreement upload → covenant extraction → manual review → compliance engine → dashboard. Synthetic data only.
- **v0.2 — Chat:** RAG retrieval chain over the extracted portfolio, with citations.
- **v0.3 — Alerting:** configurable early-warning thresholds, email/webhook delivery.
- **v0.4 — Multi-document agreements:** amendments and side letters that modify an original covenant — the same "which version is definitive" problem that shows up in any contract-reading pipeline.
- **Stretch:** fine-tuned extraction confidence scoring, so the human-review queue prioritizes the extractions most likely to be wrong.

CovenantIQ — a portfolio project demonstrating applied LLM/LangChain engineering for financial-services document intelligence. Synthetic data only. Not affiliated with, and does not process data from, any real financial institution.